

操作系统研讨课

Course: B0911011Y

卢天越

2026 春季学期



Project 5 Device Driver

2026.5.15



今日课程内容

- Project 5介绍
- Project 4及之前Project验收
- 答疑



截止P4

- 操作系统已经具备进程调度、进程通信、虚拟内存等基本功能
- SD卡可以通过bios调用来读写，且被用作页替换的空间
 - 还需要支持更多外设



Project 5概览

- 以网卡为例，实现一个设备驱动
 - 通过网络，可以发送和接收各种信息的包
- 网卡设备存在硬件约定，需要按照约定编写驱动程序
 - 通过特殊的内存地址访问网卡的DMA控制器
 - 描述符：写在内存上的一段空间，网卡硬件也会读取
 - 数据包也是通过内存空间来进行交互



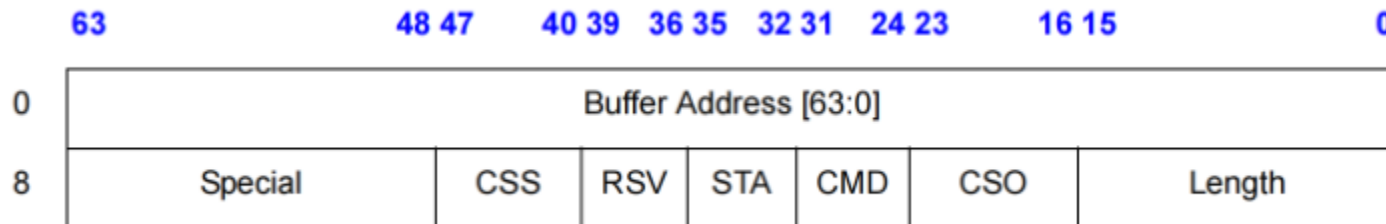
DMA过程

- 软件通过DMA寄存器向设备发出命令
 - 将要发送或接收的内存地址告诉DMA控制器，DMA就可以自行读取内存发送数据或把接收到的数据放入内存
 - 但是，DMA寄存器不够多，无法处理复杂信息
- 描述符：将一些控制信息以约定好的数据结构放在内存中
 - 再把描述符地址写入DMA寄存器



发送描述符

- E1000发送描述符
 - Buffer和length字段代表数据包地址和长度
 - CMD代表命令，初始化之后不用变
 - STA表示状态，由硬件修改供软件查询

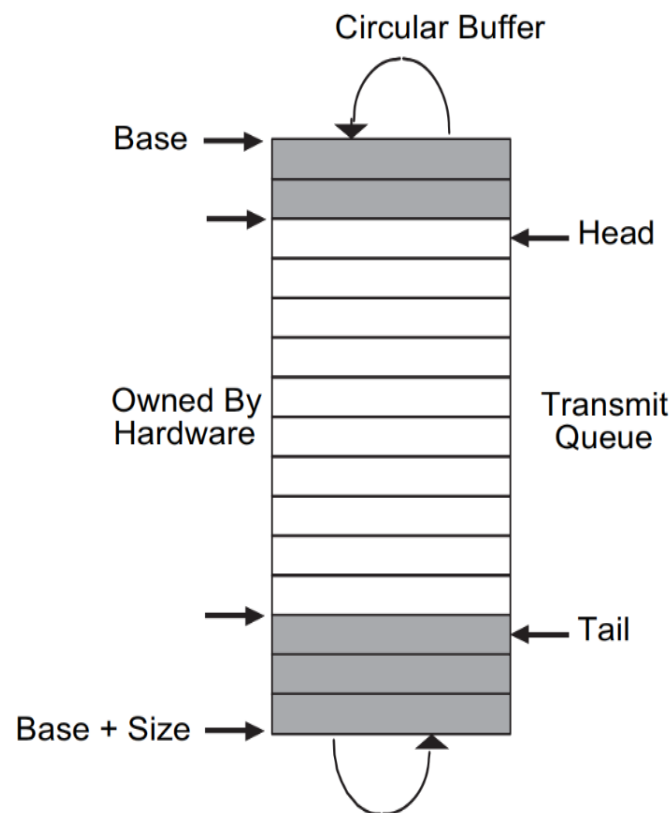


- 其他字段用不到



发送描述符循环数组

- 同时存在多个描述符以提高效率
- 多个描述符形成循环数组
 - TDBAL、TDBAH代表地址
 - TDLEN代表长度
 - 描述符固定16字节
 - TDH和TDT代表头尾指针
 - 均为MAC寄存器



发送使能

- 描述符写好之后就可以通知DMA控制器了
- 发送控制寄存器TCTL的EN位
 - 被使能且TDH不等于TDT，则启动发送
- 注意事项：所有的地址
 - 给硬件看的应为物理地址
 - 操作系统读写时应为内核虚地址



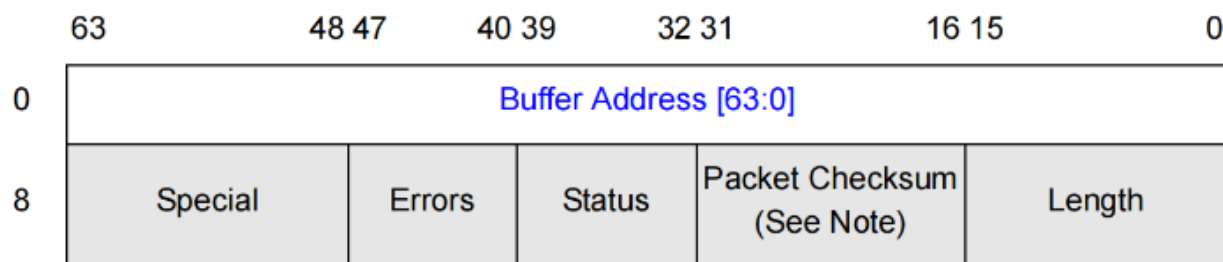
任务1：网卡初始化与轮询发送

- 实现系统调用sys_net_send，调用网卡接口发送数据包
- 通过网线与电脑连接，因此接收方需要用wireshark软件或tcpdump软件抓包
 - Windows环境需要同学们自己安装wireshark
 - Linux系统或者Linux虚拟机使用tcpdump



接收描述符

- 与发送类似
 - 灰色部分均由硬件填写
 - 字段位置不一样，Status字段更长一些



- 接收描述符也是按循环数组组织
 - 数组信息写在RDBAL、RDBAH、RDLEN等寄存器，对应T开头的发送寄存器



接收使能

- 与发送类似，使用RCTL寄存器
 - 除了EN位之外，还需要设置另外三个位
 - BAM位为1，代表支持广播帧接收
 - BSEX和BSIZE为0，代表接收缓冲区大小为2048字节
- 还需要把自身的MAC地址写进RAH、RAL寄存器
 - RAH为高位，RAL为低位

```
19 // Fixed Ethernet MAC Address of E1000
20 static const uint8_t enetaddr[6] = {0x00, 0x0a, 0x35, 0x00, 0x1e, 0x53};
```



任务2：轮询接收

- 实现系统调用sys_net_recv，运行接收测试程序recv
 - 同时跑fly小程序，代表系统未被recv卡死
- 主机使用pktRxTx小程序发包
 - 程序包已上传至国科大在线，使用方法见附带的README文档
 - 使用pktRxTx -m 1命令启动，然后在界面中输入send < 发送数据包的数目 > 命令即可
 - 数目不小于64，验收时会send多次
- 注意事项：
 - 读写描述符之后，为了保持cache一致性，需调用flush_dcachefunction函数刷数据cache



网卡中断

- 轮询接收显然存在性能问题
- A-core需要完成网卡中断
- 网卡中断
 - 在网卡发送完成或接收到一定数量的数据包之后，发中断通知CPU来处理
 - 中断处理流程与定时器中断类似



PLIC中断

- 四级中断处理：
 - 第一级：跳转到例外入口
 - 第二级：通过scause寄存器判断为外设中断
 - 第三级：通过PLIC判断为E1000中断
 - 第四级：通过E1000 MAC寄存器判断
- PLIC：平台级中断控制器
 - PLIC接入的中断需要仲裁决定哪个中断先被处理，当PLIC决定E1000网卡中断被处理时，进入中断处理第四级
 - 通过读取PLIC的claim寄存器获取中断外设的ID



E1000网卡中断

- E1000 MAC寄存器
- 需要用到的：
 - ICR: Interrupt Cause Read, 读取中断原因
 - 类似scause寄存器
 - ICS: Interrupt Cause Set, 设置中断条件
 - IMS: Interrupt Mask Set, 设置中断条件
 - 类似sstatus和sie寄存器
 - IMC: Interrupt Mask Clear, 清中断
 - 定时器中断是怎么清中断的?



任务3：有网卡中断的收发

- 仅A-core需要完成
- 完成中断处理函数e1000_handle_txqe 和 e1000_handle_rxdmt0
- 完成第三级PLIC中断处理函数
 - 注意PLIC也需要用completion操作清中断
- 使能网卡中断



设备驱动调试注意事项

- 设置寄存器

- 出错之后不会跑飞或报错，只会默默不发送或不接收，任何寄存器或描述符都有可能出错
- MAC寄存器在gdb上看不到，需要打印log

```
uint32_t plic_claim(void)
{
    struct plic_handler *handler = &plic_handlers;
    void *claim = handler->hart_base + CONTEXT_CLAIM;
```

```
    return readl(claim);
}
```

PLIC操作已经写好，直接调用即可

```
void plic_complete(int hwirq)
```

```
{
```

```
    struct plic_handler *handler = &plic_handlers;
```

```
    writel(hwirq, handler->hart_base + CONTEXT_CLAIM);
```

```
}
```

```
static void e1000_configure_rx(void)
```

```
{
```

```
    /* TODO: [p5-task2] Set e1000 MAC Address to RAR[0] */
```

```
    /* TODO: [p5-task2] Initialize rx descriptors */
```

```
    /* TODO: [p5-task2] Set up the Rx descriptor base address and length */
```

```
    /* TODO: [p5-task2] Set up the HW Rx Head and Tail descriptor pointers */
```

```
    /* TODO: [p5-task2] Program the Receive Control Register */
```

```
    /* TODO: [p5-task4] Enable RXDMT0 Interrupt */
```

```
}
```



任务4：可靠传输

- 仅C-core需要完成
- 简化的传输层协议：
 - 在包头中包含发送、确认、重传等标记
 - pktRxTx小程序已经支持，需要同学们按照任务书指明的格式进行处理和发送
- 完整的接收一个大文件并算出正确的校验和
 - pktRxTx小程序发送速度过快，大于接收速度时，一定会产生丢包，需要接收方发送重传包
 - 一旦没有收到对应的确认包，小程序就会停止发送



其他注意事项

- 做实验时请尽量关闭主机上的程序，它们有可能不停的发送网络包
- **QEMU调试**时是从QEMU向虚拟机主机发包，所以收包应该是在**虚拟机上使用tcpdump**
- Wireshark和pktRxTx小程序，都需要注意网卡名称，只有正确的网卡才能看到包
 - Windows可以在控制面板的网络设置里看
 - Linux可以用ifconfig命令看



P5 Design review

- 介绍你的设计，覆盖以下问题：
 - 说明发送和接收描述符的格式
 - 说明从网卡初始化到发送使能或接收使能的全过程
 - 介绍网卡中断的处理流程



谢谢!

下节课：5月22日，P5 review
Any questions?

